

SYNTHETIX · SECURITY REVIEW

Synthetix

Auto Compound LP.

A security review of Auto Compound LP.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	Auto Compound LP	Guardian	April 14 – May 7, 2025	May 14, 2025

§ 00

Contents.

104 pages — overview, confidence ranking, scope, 80 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 65 findings	P. 13
	Remediation Review · 15 findings	P. 87
§ 05	Disclaimer	P. 103
§ 06	About Guardian	P. 104

§ 01

Overview.

Guardian reviewed Auto Compound LP across 2 review rounds. The final report includes 1 Critical, 4 High, 14 Medium, 61 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/auto-compound-lp, Synthetixio/infra
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Auto Compound LP	19433ae94f66
PREPARED BY	
Guardian	
AUDIT WINDOW	
April 14 – May 7, 2025	
FINAL REPORT	
May 14, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	1	0	0	0	1
■ High	4	0	0	0	4
■ Medium	14	0	9	2	3
■ Low	61	0	45	0	16
■ Informational	0	0	0	0	0
All severities	80	0	54	2	24

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Auto Compound LP in `Synthetixio/auto-compound-lp`, `Synthetixio/infra`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Unauthorized Rewards Claiming Leads To Loses	Rewards	CRITICAL	RESOLVED	p. 13
H-01	Anyone Can Update Users userLastDeposited	Access Control	HIGH	RESOLVED	p. 14
H-02	Transfer Of SLP Shares Is Not Compatible With ERC2771	Logical Error	HIGH	RESOLVED	p. 16
H-03	USDC Rewards Break The AutoCompoundingLP	Unexpected Behavior	HIGH	RESOLVED	p. 17
H-04	Keeper : Swapping Unconfigured Tokens Including SNX	Logical Error	HIGH	RESOLVED	p. 19
M-01	Penalty Manipulation Allows Attacker To Steal Depositor's Funds	Frontrunning	MEDIUM	ACKNOWLEDGED	p. 20
M-02	Migration Doesn't Check For Locked Collateral	DoS	MEDIUM	ACKNOWLEDGED	p. 21
M-03	Keeper : Unauthenticated Public API Gateway	Access Control	MEDIUM	PARTIALLY RESOLVED	p. 22
M-04	Faulty USDC Delta Check Can Be Bypassed	Logical Error	MEDIUM	ACKNOWLEDGED	p. 23
M-05	Keeper : Reverts On USDC-To-USDC Swap Attempt	Logical Error	MEDIUM	RESOLVED	p. 24
M-06	Keeper : Unvalidated ODOS Swaps And Private Key Exposure Risk	Validation	MEDIUM	ACKNOWLEDGED	p. 25

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-07	Keeper : Hard-coded Secrets & RPC Key	Secret Management	MEDIUM	RESOLVED	p. 26
M-08	Keeper : Plain-Text Private-Key Handling	Secret Management	MEDIUM	RESOLVED	p. 27
M-09	Keeper : No Throttling / Rate Limits	DoS	MEDIUM	PARTIALLY RESOLVED	p. 28
M-10	Keeper : Lambda SG Allows All Outbound	Network Egress	MEDIUM	ACKNOWLEDGED	p. 29
M-11	Keeper : WAF Not Enabled	Web Layer Protection	MEDIUM	ACKNOWLEDGED	p. 30
L-01	Clearing Debt May DOS The Migration	Rounding	LOW	ACKNOWLEDGED	p. 31
L-02	Too Many Locks Can Block Migrations	Informational	LOW	ACKNOWLEDGED	p. 33
L-03	stataUSDC Should Not Be Added As Reward	Warning	LOW	ACKNOWLEDGED	p. 34
L-04	totalAssetsStata Can Be Slightly Inflated	Rounding	LOW	ACKNOWLEDGED	p. 35
L-05	WrapperModule Rounding Issues	Rounding	LOW	ACKNOWLEDGED	p. 36
L-06	Loss Of Rewards From Removed Distributor	Rewards	LOW	ACKNOWLEDGED	p. 38
L-07	DoS Due To Synthetix maxWrappableAmount Limit	DoS	LOW	ACKNOWLEDGED	p. 39
L-08	Loss Of Rewards When Updating rewardsRegistry	Rewards	LOW	ACKNOWLEDGED	p. 41
L-09	DoS In Reward Claiming Due To Strict Slippage	DoS	LOW	ACKNOWLEDGED	p. 42
L-10	Unused Token Approvals To ODOS Pose Security Risk	Logical Error	LOW	ACKNOWLEDGED	p. 44
L-11	Old Rewards Registry Retains Permissions After Replacement	Logical Error	LOW	ACKNOWLEDGED	p. 45

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-12	AAVE Deposit/Redeem Limits May Affect stataUSDC Operations	Error-Handling	LOW	ACKNOWLEDGED	p. 46
L-13	Vault Deviates From ERC4626	Warning	LOW	ACKNOWLEDGED	p. 47
L-14	RewardsDistributionRegistry Lacks Flexibility	Warning	LOW	RESOLVED	p. 48
L-15	Issues With Reward Tokens Array Handling	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 49
L-16	Penalty Rounding In Favor Of The User	Rounding	LOW	RESOLVED	p. 50
L-17	Use safeApprove() & safeTransfer()	Best Practices	LOW	ACKNOWLEDGED	p. 51
L-18	Keeper : Missing Validation On Rewards Return Value	Validation	LOW	RESOLVED	p. 52
L-19	Keeper : getRewards Skips Tokens Held In Vault Balance	Logical Error	LOW	RESOLVED	p. 53
L-20	Keeper : Incomplete Error Handling	Error-Handling	LOW	RESOLVED	p. 54
L-21	Keeper : assembleOdosQuote May Use Expired Or Invalid Quotes	Error-Handling	LOW	RESOLVED	p. 55
L-22	Keeper : No Monitoring Of Keeper ETH	Error-Handling	LOW	ACKNOWLEDGED	p. 56
L-23	Keeper : Non-Checksummed Addresses	Logical Error	LOW	RESOLVED	p. 57
L-24	Keeper : Potential Sandwich Risk	Warning	LOW	ACKNOWLEDGED	p. 58
L-25	Keeper : Default Slippage May Misprice Reward	Logical Error	LOW	RESOLVED	p. 59
L-26	Migration Omits Synthetix Reward Claiming	Rewards	LOW	ACKNOWLEDGED	p. 60
L-27	Rewards Discount Impacts Share Calculations	Informational	LOW	ACKNOWLEDGED	p. 61

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-28	Tokens With Invalid Prices Are Treated As 0	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 62
L-29	Unconditional Rounding In <code>_repay()</code>	Rounding	LOW	ACKNOWLEDGED	p. 63
L-30	Cached Assets Don't Account For Redeem Penalty	Math	LOW	ACKNOWLEDGED	p. 64
L-31	Liquidations Break The Vault	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 65
L-32	Donator Can Undelegate Vault Collateral	Informational	LOW	ACKNOWLEDGED	p. 66
L-33	Locked Market Can Block Redeem	Informational	LOW	ACKNOWLEDGED	p. 67
L-34	Unnecessary Reward Operation	Informational	LOW	ACKNOWLEDGED	p. 68
L-35	Alchemy API Key Exposed	Warning	LOW	RESOLVED	p. 69
L-36	Keeper : Dead Code / TODOs	Best Practices	LOW	ACKNOWLEDGED	p. 70
L-37	Keeper : Hard-coded Resource Identifiers	Best Practices	LOW	ACKNOWLEDGED	p. 71
L-38	Keeper : Non-optimal Lambda Sizing	Best Practices	LOW	ACKNOWLEDGED	p. 72
L-39	Keeper : Opaque Cross-Stack Imports	Best Practices	LOW	ACKNOWLEDGED	p. 73
L-40	Keeper : Sensitive Request/Response Logging	Best Practices	LOW	ACKNOWLEDGED	p. 74
L-41	Keeper : Short Log Retention	Best Practices	LOW	RESOLVED	p. 75
L-42	Keeper : Inefficient Secret Retrieval	Best Practices	LOW	RESOLVED	p. 76
L-43	Keeper : RemovalPolicy DESTROY On Logs	Best Practices	LOW	RESOLVED	p. 77
L-44	Keeper : External Calls Lack Timeout & Retry	Best Practices	LOW	RESOLVED	p. 78

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-45	Keeper : Missing Input Validation	Validation	LOW	RESOLVED	p. 79
L-46	Keeper : Over-Permissive / Implicit Lambda Role	IAM	LOW	RESOLVED	p. 80
L-47	redeem Revert Due To Flash Loan Underfunding	DoS	LOW	ACKNOWLEDGED	p. 81
L-48	Keeper :Outdated AWS CDK And Deprecated SDK	Best Practices	LOW	RESOLVED	p. 83
L-49	Trust Assumptions	Informational	LOW	ACKNOWLEDGED	p. 84

REMEDIATION REVIEW

M-01	Stuck Rewards in RewardsDistributionRegistry	Logical Error	MEDIUM	ACKNOWLEDGED	p. 87
M-02	Keeper Ineffective /tmp Caching Logic Causes Crash on Cold Start	IAM	MEDIUM	ACKNOWLEDGED	p. 88
M-03	Keeper Wildcard SecretsManager Policy	Access Control	MEDIUM	ACKNOWLEDGED	p. 90
L-01	Interface Mismatch for migrateLP Signature	Logical Error	LOW	ACKNOWLEDGED	p. 91
L-02	L-16 Partially Fixed : Rounding Penalty	Logical Error	LOW	ACKNOWLEDGED	p. 92
L-03	Inconsistent Calculations in case of USDC depeg.	Logical Error	LOW	ACKNOWLEDGED	p. 93
L-04	Unfixed H-03	Warning	LOW	ACKNOWLEDGED	p. 94
L-05	Alchemy API key can still be accessed	Logical Error	LOW	ACKNOWLEDGED	p. 95
L-06	ODOSRouter Max Approvals : Risk Increased	Warning	LOW	ACKNOWLEDGED	p. 96
L-07	Keeper Invalid handler signature	Logical Error	LOW	ACKNOWLEDGED	p. 97

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	Keeper RPC-Key is still hardcoded	Warning	LOW	ACKNOWLEDGED	p. 98
L-09	Keeper Slippage Tolerance Not Passed to ODOS in Quote Request	Logical Error	LOW	ACKNOWLEDGED	p. 99
L-10	Keeper L21 Partially Fixed: Missing Error Handling for assemble0dosQuote Response	Validation	LOW	ACKNOWLEDGED	p. 100
L-11	Keeper Missing Input Validation	Validation	LOW	ACKNOWLEDGED	p. 101
L-12	Keeper All addresses aren't checksummed	Logical Error	LOW	ACKNOWLEDGED	p. 102

CRITICAL **C-01**

RESOLVED

Unauthorized Rewards Claiming Leads To Loses

CATEGORY Rewards
LOCATION RewardsDistributionRegistry.sol: 200
REVIEW Main Review

DESCRIPTION

`RewardsDistributionRegistry.claimRewards()` uses `claimRewardsFromDistributor()` to claim the rewards from the Synthetix core system. Once done, the tokens will be transferred to the `RewardsDistributionRegistry` contract. The rest of the `claimRewards` function will perform unwrapping (if needed) and will send the newly claimed amount to the `msg.sender`, which is expected to be the `AutoCompoundLP`.

However, the `claimRewardsFromDistributor` function is public which means anyone can call it, and claim the rewards.

The Synthetix core will send them again to the `RewardsDistributionRegistry` with no way to get them out of there.

Even if `claimRewards()` is called, as already mentioned, only the newly claimed amount will be sent to the user.

In result, the rewards are stuck and the vault loses its revenue. The attack can be repeated again and again for as cheap as paying gas fees.

It's expected that all users can use `RewardsDistributionRegistry` to claim their rewards, not just the `AutoCompoundLP`, therefore they are also impacted by this vulnerability.

RECOMMENDATION

Make `claimRewardsFromDistributor()` internal

PROOF OF CONCEPT <https://gist.github.com/ZdravkoHr/3f70f146eadb4d97d17dd5d2128a6066>

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/auto-compound-lp/pull/56>

HIGH **H-01**

RESOLVED

Anyone Can Update Users userLastDeposited

CATEGORY Access Control
LOCATION LPMigration.sol: 108
REVIEW Main Review

DESCRIPTION

Every time a user receives `AutoCompoundLP` tokens in result of a deposit, their `userLastDeposited` mapping is updated with the timestamp at which the deposit happen. This value is then used to determine two things:

- whether the tokens of the user can be transferred (they can't if a given period of time hasn't passed since the last deposit time)
- calculate the amount of penalty the user should pay if they decide to redeem their tokens (the sooner the deposit, the greater the penalty)

The `AutoCompoundLP.deposit()` function checks that the specified `receiver` is the `msg.sender` if the function is not invoked by the `LPMigrator`. This protects users from becoming a target of a small deposit with the goal of resetting their `userLastDeposited` mapping.

If we take a look at `LPMigrator.migrate()`, which is responsible for calling `AutoCompoundLP.deposit()`, we will see that the `receiver` passed is never verified - it can be any arbitrary address. This allows malicious actors to bypass the `receiver` restriction in the following way:

- create a brand new account with no debt and no collateral
- send a small amount of `USDC` to `LPMigrator` and a desired `recipient`
- The `recipient` receives the `AutoCompoundLP` tokens and their `userLastDeposited` mapping is updated
- They cannot transfer their tokens until the period passes and have to also pay the full penalty amount on redeem

These steps can be repeated again and again.

RECOMMENDATION

Don't allow passing arbitrary receiver to `LPMigration.migrate()`

H-01 Anyone Can Update Users userLastDeposited

CONTINUED

```
- IAutoCompoundLP(autoCompoundVault).deposit(usdcBalance, receiver);  
+ IAutoCompoundLP(autoCompoundVault).deposit(usdcBalance, msgSender);
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/auto-compound-lp-team1/pull/1>**— RESOLUTION**

Status: Resolved.

<https://github.com/Synthetixio/auto-compound-lp/pull/56>

HIGH **H-02**

RESOLVED

Transfer Of SLP Shares Is Not Compatible With ERC2771

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L855
REVIEW	Main Review

DESCRIPTION

SNX intends to allow ERC2771 forwarders to perform transfers of SLP shares. However, since it uses its own `ERC2771Context._msgSender()` implementation and invokes the `transfer` function of `ERC20Upgradeable` from OpenZeppelin—which relies on its inherited `ContextUpgradeable`—the `msgSender()` considered during the transfer is always `msg.sender`.

As a result, all transactions initiated as meta-transactions for transfer will fail, since the transfer will be attempted from `msg.sender`, i.e., the forwarder contract itself.

```
function transfer(
    address to,
    uint256 value
) public override returns (bool) {
    block.timestamp - userLastDeposited[ERC2771Context._msgSender()] <
    decayingRedemptionPenaltyDuration
} {
    revert TransferBlocked();
}
return super.transfer(to, value); // @audit doesnt consider context, would attempt transfer
from forwarder
}
```

RECOMMENDATION

Consider overriding `_msgSender()` of context upgradable

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/auto-compound-lp/pull/56>

HIGH **H-03**

RESOLVED

USDC Rewards Break The AutoCompoundingLP

CATEGORY Unexpected Behavior

LOCATION RewardsDistributionRegistry.sol: 261; RewardsDistributionRegistry.sol: 165

REVIEW Main Review

DESCRIPTION

`RewardsDistributionRegistry` has a function `claimRewards()`, as well as `getRewards()` function that returns the amount of rewards that can be claimed.

Both `claimRewards()` and `getRewards()` have a `totalClaimed` variable showing how much rewards of the given token have been claimed. It comes from `claimRewardsFromDistributor()` or `claimRewardsFromDistributor()`. The amount returned and therefore `totalClaimed` is always gonna be in 18 decimals precision, as we can see in the following vault logic [here] (<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/RewardsManagerModule.sol#L172-L179>) and [here] (<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/storage/Vault.sol#L204>).

The problem is that if `rewardIsSynth == false`, `rewardAmount = totalClaimed` will be executed, but all of the code afterwards assumes that `rewardAmount` is in the decimals of the rewardToken.

For example, `claimRewards()` will revert for USDC because it tries to transfer `rewardAmount` of USDC which is in 18 decimals, resulting in DOS of the whole `claimRewards()` function.

`getUsdValueOfRewards()` makes a conversion by multiplying by $1e18$ and dividing by $10^{(\text{rewardToken.decimals})}$. For USDC, this results in a value with 30 decimals. This will report extremely large value to `AutoCompoundLP` and will completely break it.

RECOMMENDATION

Convert the `rewardAmount` to have the same decimal precision as the reward token on both lines:

`RewardsDistributionRegistry.sol:L261``RewardsDistributionRegistry.sol:L165`

H-03 USDC Rewards Break The AutoCompoundingLP

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/auto-compound-lp-team1/blob/80f5f3410456542f0e71c007063d071bc4cbfbca/contracts/test/AutoCompoundLP.t.sol#L609-L627>

— RESOLUTION

Status: Resolved.

HIGH **H-04**

RESOLVED

Keeper : Swapping Unconfigured Tokens Including SNX

CATEGORY	Logical Error
LOCATION	main.ts, config.ts
REVIEW	Main Review

— DESCRIPTION

The current script swaps SNX as well, which causes a spike in `totalAssetsStata`, potentially enabling a sandwich attack.

The SNX team does not intend to swap SNX received as rewards and instead hardcodes its price as 0 in the calculation of `totalAssetsStata`.

However, the keeper swaps all tokens returned from `getRewards`, including SNX—even when `min` and `max` are not defined for the token. The keeper simply ignores tokens without defined `min` and `max`, but still proceeds to swap them.

A further issue with this approach is that if any unexpected token is received as a reward (i.e., not currently configured), the keeper will attempt a full swap of the entire amount, potentially causing significant slippage.

Additionally, this logic can lead to an unnecessary swap attempt of `stataUSDC`, which should not be swapped.

— RECOMMENDATION

Only swap tokens that are explicitly listed in the `config.ts` file and have both `min` and `max` defined.

Log any ignored tokens.

If the ignored token is something other than SNX, trigger an alert so the SNX team can review and update the config accordingly.

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

Penalty Manipulation Allows Attacker To Steal Depositor's Funds

CATEGORY	Frontrunning
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L276
REVIEW	Main Review

— DESCRIPTION

A penalty in the `AutoCompoundLP` contract allows an attacker to manipulate the ratio of total assets to total supply, enabling an inflation attack.

By performing a strategic deposit followed by a near-complete redemption (leaving only one share), the attacker can distort the asset-to-share ratio, causing subsequent depositors to receive zero shares for their deposits.

The attacker deposits a large amount and redeems all but one share, triggering a penalty and setting the total supply to 1 while maintaining total assets equal to the penalty.

This creates a state where a second depositor's assets are absorbed without receiving any shares, as their deposit amount is insufficient to overcome the manipulated ratio and penalty.

The attacker can further dilute the ratio and redeem the assets after a decay window, potentially stealing the second depositor's funds.

— RECOMMENDATION

Consider minting dead shares equivalent to min delegation amount inside initialise.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/d6c69539ff8f7e50dde3fa25f716a1e4>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Migration Doesn't Check For Locked Collateral

CATEGORY DoS
LOCATION LPMigration.sol: 317; LPMigration.sol: 338
REVIEW Main Review

DESCRIPTION

LPMigration withdraws snxUSD from the user's position in `_withdrawAvailableSnxUsd()` and `synth` in `_undelegateAndUnwrapSynth()`. The amount to be withdrawn is fetched from `getAccountAvailableCollateral()`. However, this doesn't account for any active collateral locks.

For example, user has 500 available collateral, but 100 of them is locked, which means at most 400 can be withdrawn. The migrator will try withdrawing all of it (500) and the transaction will revert, resulting in inability for the user to migrate using the contract until their lock expires.

RECOMMENDATION

Take into consideration the locked amount when computing how much collateral is to be withdrawn.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

PARTIALLY RESOLVED

Keeper : Unauthenticated Public API Gateway

CATEGORY	Access Control
LOCATION	lp-vault-keeper.ts
REVIEW	Main Review

— DESCRIPTION

The POST /lp-vault-keeper endpoint is exposed without any authorizer, API-key, or IAM requirement, enabling anyone on the Internet to invoke sensitive Lambda logic.

Even though the Lambda runs in private subnets, the API Gateway lives outside the VPC; it exposes a public HTTPS URL and then calls the Lambda over AWS's internal channel. The VPC blocks direct inbound traffic to the function, but it does nothing to hide the Gateway itself, so anyone who knows the URL can still invoke the endpoint

— RECOMMENDATION

Require authentication — add a JWT/Lambda authorizer, IAM auth, or at minimum an API-key with usage-plan limits.

— RESOLUTION

Status: Partially resolved.

MEDIUM **M-04**

ACKNOWLEDGED

Faulty USDC Delta Check Can Be Bypassed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L340-L341
REVIEW	Main Review

— DESCRIPTION

The SNX team assumes that checking if `(balanceAfter - balanceBefore) < assetsReceived` is sufficient to prevent misuse in case of keeper compromise.

They specifically mention this safeguard to prevent a scenario where the keeper sets themselves as the receiver:

> "The main way that we saw this being possible is swapping tokens out to a separate to address. This is accounted for by the smart contract checking the returned value of the odos swap—which is the USDC received—and asserting that it equals the difference in USDC balance before and after the swap."

[Notion Link](<https://www.notion.so/Overview-and-security-concerns-for-Audit-1c10a80a459e8016a2adccd55fb5b8e1?pvs=21>)

However, this assumption is flawed. `assetsReceived` does not validate the output token itself.

A malicious keeper could perform a swap into a token with a lower value per unit than USDC, misleading the contract into believing USDC was received.

— RECOMMENDATION

If the goal is to validate `assetsReceived`, ensure the output token is explicitly checked.

That said, even with such validations, a malicious keeper could still find other ways to extract funds, so further defense-in-depth mechanisms should be considered.

For example: M-06

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

RESOLVED

Keeper : Reverts On USDC-To-USDC Swap Attempt

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/main.ts#L100-L101
REVIEW	Main Review

— DESCRIPTION

Whenever the keeper receives USDC as rewards, it will revert.

This happens because the keeper attempts to swap USDC to USDC, and the Odos API reverts when the input token is the same as the output token.

— RECOMMENDATION

Consider skipping USDC during `inputTokens` filtering.

— RESOLUTION

Status: Resolved.

MEDIUM **M-06**

ACKNOWLEDGED

Keeper : Unvalidated ODOS Swaps And Private Key Exposure Risk

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The keeper script, and therefore the vault, will execute whatever swap is quoted by the ODOS API.

Since ODOS is an external dependency, blindly trusting its output is risky.

Additionally, the keeper uses an EOA.

If the private key leaks from any developer machine, the contract functions would be exposed.

— RECOMMENDATION

Add validations for swap quotes returned by ODOS. These checks must execute within 60 seconds to ensure the quote does not expire.

The validation could involve simulating the `claimSwapCompound` function and checking the delta in `totalAssetsStata`. If the delta exceeds a certain threshold, the swap should be rejected.

For improved key management and security, consider adding this validation for delta between before and after keeper actions at the smart contract level.

Since SNX already tracks `totalAssetsStata` and does not expect large deviations from keeper actions, this validation can be incorporated easily.

Implementing the validation at the contract level would also safeguard SNX in case the keeper script or its private key is ever compromised.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

RESOLVED

Keeper : Hard-coded Secrets & RPC Key

CATEGORY Secret Management
LOCATION config.ts / utils.ts
REVIEW Main Review

DESCRIPTION

`SECRET_ARN` and an Alchemy RPC key are committed in plaintext. Attackers with repo access can extract production secrets or drain paid API quotas.

RECOMMENDATION

Store secrets exclusively in AWS Secrets Manager / SSM Parameter Store and reference them via environment variables blocked by pre-commit secret-scans.

RESOLUTION

Status: Resolved.

MEDIUM **M-08**

RESOLVED

Keeper : Plain-Text Private-Key Handling

CATEGORY Secret Management
LOCATION utils.ts / main.ts
REVIEW Main Review

DESCRIPTION

`getPrivateKey()` returns the raw string and downstream code could log or crash with key in memory.

RECOMMENDATION

Return an AWSKMS-wrapped Signer (e.g., `ethers.Wallet.fromEncryptedJson`) and zero sensitive buffers after use.

RESOLUTION

Status: Resolved.

MEDIUM **M-09**

PARTIALLY RESOLVED

Keeper : No Throttling / Rate Limits

CATEGORY DoS
LOCATION lp-vault-keeper.ts
REVIEW Main Review

— DESCRIPTION

API Gateway lacks usage plans and per-method throttling; attackers can exhaust Lambda concurrency and upstream quotas.

— RECOMMENDATION

Apply a usage plan (e.g., 100 req/min) and set Reserved Concurrency on the Lambda.

— RESOLUTION

Status: Partially resolved.

MEDIUM **M-10**

ACKNOWLEDGED

Keeper : Lambda SG Allows All Outbound

CATEGORY Network Egress
LOCATION lp-vault-keeper.ts
REVIEW Main Review

DESCRIPTION

`allowAllOutbound: true` permits the lambda function to connect anywhere, creating an exfiltration path if compromised and malicious code was injected into the lambda.

RECOMMENDATION

Restrict egress to 443 toward approved FQDNs/CIDRs (Alchemy, Odos, AWS APIs) via prefix-lists or explicit rules.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Keeper : WAF Not Enabled

CATEGORY Web Layer Protection
LOCATION lp-vault-keeper.ts
REVIEW Main Review

DESCRIPTION

The public API is not fronted by AWS WAF, exposing it to OWASP Top-10 attacks and volumetric abuse.

RECOMMENDATION

Attach AWS WAFv2 web-ACL to the API stage; enable managed rules (`AWSManagedRulesCommonRuleSet` , `BotControl`).

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Clearing Debt May DOS The Migration

CATEGORY Rounding

LOCATION <https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/LPMigration.sol#L238-L240>

REVIEW Main Review

DESCRIPTION

In `LPMigration._clearDebt()`, the `usdcAmount` needed to repay the debt is increased by 1 to account for rounding errors. Notice that this is not rounding up, it happens even if the result is a round number.

Because of this, a user may end up in a situation where he has X amount of debt and X amount of `USDC` in their wallet, but the code will calculate the `usdcAmount` as `X + 1`.

Because of this, the X will be repaid by the user's wallet and the 1 will be taken as a flashloan.

However, the user's X is enough to repay the whole debt and as result it will be erased.

This will make `debtAmount - repayableDebt = 0`, and `executeOperation()` will try selling 0 sUSD.

Because the `sUSD.burn()` function reverts if the amount to be burned is 0, the whole transaction will be unsuccessful.

RECOMMENDATION

You can implement one of the following:

1. Round the `usdcAmount` up only if needed
2. Skip the flashloan if the new debt is 0

```
debt = ICoreProxy(CoreProxy).getPositionDebt(
    accountId,
    poolId,
    collateral
);
if (debt == 0) {
    return;
}
_aavePool.flashLoanSimple(
    address(this),
```

L-01 Clearing Debt May DOS The Migration

CONTINUED

```
$USDC,  
usdcAmount - userUxdcBalance,  
abi.encode(  
  accountId,  
  debtAmount - repayableDebt,  
  debtAmount - repayableDebt,  
  collateral  
)  
,  
0  
);
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/auto-compound-lp-team1/blob/a604da96d3ca45abf3d17131c6eefbe161952aaa/contracts/test/LPMigration.t.sol#L137-L211>

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Too Many Locks Can Block Migrations

CATEGORY Informational
LOCATION LPMigration.sol
REVIEW Main Review

DESCRIPTION

`LPMigration` uses `CollateralModule.deposit()` when clearing the positive user debt. This action processes all locks of the user (`9999999999999999` is used as `count` for `cleanAccountLocks`). Because of this, accounts with too many locks may require too much gas to be migrated, potentially even blocking the migration.

RECOMMENDATION

Inform users about locks increasing the cost of the migration.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

stataUSDC Should Not Be Added As Reward

CATEGORY Warning
LOCATION AutoCompound.sol
REVIEW Main Review

— DESCRIPTION

`AutoCompoundLP.totalAssetsStata()` adds the value of the reward tokens held by that contract towards all the available assets. Should `stataUSDC` be added as a reward tokens, `_convertToAssetsStata` calculations will return wrong results. For example, when users deposit, their `USDC` is turned into `stataUSDC` and then `_convertToAssetsStata()` is called. Having `stataUSDC` itself as a reward token will lead to inflation in the amount of available assets and thus the user receiving wrong amount.

— RECOMMENDATION

Don't add `stataUSDC` as reward token for as long as the code performs the assets calculation like it does now.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

totalAssetsStata Can Be Slightly Inflated

CATEGORY Rounding
LOCATION AutoCompoundLP.sol: 205
REVIEW Main Review

DESCRIPTION

`AutoCompoundLP.totalAssetsStata()` adjusts the available assets amount by converting the absolute value of `netNonNativeAmount` to `stataUSDC` by dividing by the `rate()`.

In cases when `netNonNativeAmount` is negative, the `mulDiv` will round the result up. For example, if the debt is -5.5, the function will round it as 5, not 6 and in result `totalAssetsStata` will be inflated.

RECOMMENDATION

Consider rounding in the opposite direction if `netNonNativeAmount` is negative. This will cause precision loss in the other direction, but it's generally better to overestimate loses compared to underestimating them.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

WrapperModule Rounding Issues

CATEGORY Rounding
LOCATION Global
REVIEW Main Review

DESCRIPTION

`WrapperModule.wrap()` and `WrapperModule.unwrap()` take an amount of tokens to `wrap/unwrap`, applies a fee to it and returns the result in the appropriate token. For the scope of this audit, the fees are assumed to always be 0. However, let's look at the way the amounts are computed.

```
uint256 usdAmount = baseAmountD18.mulDecimal(synthPrice);  
synthAmount = usdAmount.divDecimal(synthPrice);
```

These 2 lines are taken from `wrap`, but they are the same in `unwrap` (the names are different). Let's replace `mulDecimal` and `divDecimal`. Then we got

```
uint256 usdAmount = baseAmountD18 * synthPrice / 1e18;  
synthAmount = usdAmount * 1e18 / synthPrice;
```

Because of the first division, `usdAmount` can experience a loss up to (but not including) `1`. This loss is then amplified by the multiplication with `1e18 / synthPrice`, resulting in a maximum possible loss of up to `floor(1e18 / synthPrice + 1)` (+1 because of the division on the second line). For `synthPrice > 1e18` the maximum loss should be capped to `1`, but the lower the price is, the bigger the potential precision loss experienced.

To handle that, `RewardsDistributionRegistry._unwrappedAmount()` subtracts 1 from the expected received amount. As mentioned above, this should be fine for `synthPrice > 1e18`, but can result in reverting transactions if the price drops below that.

Now let's look at the `[wrap()]`(<https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L532>) that happens in `_increasePosition`. The synth market used is the `synthIdStataUSDC`. The `synthAmount` expected to be received doesn't account for the precision loss. At the moment of writing, `[indexPrice]` (<https://usecannon.com/packages/synthetix-spot-market/latest/8453-andromeda/interact/synthetix-spot-market/SpotMarketProxy/0x18141523403e2595D31b22604AcB8Fc06a4CaA61#selector-0xe450d1f2>) of that market returns a value which has its last 12 digits as 0s. If this property stays true, then there is no need to account for the precision loss because

L-05 WrapperModule Rounding Issues

CONTINUED

the converted `synthIdStataUSDC` will also have its last 12 digits as 0s which means there is no precision loss at all. However, if the price were to change its format, then the code should account for that by adjusting the slippage tolerance as well.

There are 2 more types of `wrap/unwrapped()` that haven't been discussed:

- `unwrap synthIdStataUSDC` - there is no need to account for the precision loss here because after it happens, the amount is divided by `1e12`. This means the part where the precision loss happened is irrelevant. Keep in mind that if the decimals of the `stataUSDC` change (highly unlikely), you may need to adjust the expected returned amount as well.
- `wrap/unwrap synthIdUSDC` - the price of this market is hardcoded to `1e18`, which means that when the 2 lines of code in this report are executed, there won't be a precision loss.

— RECOMMENDATION

Consider if any of the behaviors mentioned in the report should be taken into account and if so, change the slippage protections accordingly.

PROOF OF CONCEPT <https://gist.github.com/ZdravkoHr/439f479a248574e75a167c52a63ff1c5>

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Loss Of Rewards From Removed Distributor

CATEGORY	Rewards
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team2/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/RewardsDistributionRegistry.sol#L118
REVIEW	Main Review

DESCRIPTION

When `removeDistributor()` is called, it will remove the distributor from `distributorData` and `distributors`. This will make any rewards that have accrued from that distributor and not been claimed yet to be lost. This happens because distributors loop through `distributors`, and will no longer be able to claim them.

RECOMMENDATION

Prior to deleting the data for a distributor, claim the rewards from that distributor first from `AutoCompoundLP.sol`.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

DoS Due To Synthetix maxWrappableAmount Limit

CATEGORY	DoS
LOCATION	AutoCompoundLP.sol: 532
REVIEW	Main Review

DESCRIPTION

The `AutoCompoundLP` contract's core functions, `deposit` and `compound`, rely on the internal `_increasePosition` function. This function wraps `stataUSDC` into `synthStataUSDC` by calling `IWrapperModule.wrap` on the Synthetix Spot Market (`spotMarketProxy`).

The Synthetix `wrap` function enforces a market-wide `maxWrappableAmount` limit for each specific synth market. For `synthStataUSDC` (market ID `3`), this limit is currently set at 50 million units, represented as `50000000000000000000000000` (5e25) when scaled to 18 decimals, according to Synthetix V3 configuration (e.g., found in `meta.json` within node modules).

If this market-wide limit (`maxWrappableAmount`) is reached due to aggregate wrapping activity across all protocols and users interacting with the Synthetix `synthStataUSDC` market, subsequent calls to `wrap` for market ID `3` will revert.

This reversion will directly cause failures in:

1. `deposit` transactions initiated by users attempting to add liquidity to the `AutoCompoundLP` vault.
2. `compound` transactions initiated by keepers attempting to reinvest rewards. This, in turn, can cause `redeem()` to revert. This will occur because the calculation for assets takes the rewards into consideration. Therefore, a user may get an overestimated amount of USDC as a reward, that can not be sent to them due to `compound()` reverting.

This leads to a Denial of Service (DoS) for these essential vault operations. The `AutoCompoundLP` vault's ability to accept new deposits and to auto-compound rewards becomes entirely dependent on the available capacity within the external Synthetix protocol's wrapper limits for `synthStataUSDC`.

RECOMMENDATION

L-07 DoS Due To Synthetix maxWrappableAmount Limit

CONTINUED

If this limit proves to be a frequent bottleneck for the `AutoCompoundLP` vault's operation, the recommended long-term solution is to engage with Synthetix governance to request an increase in the `maxWrappableAmount` for the `synthStataUSDC` market (ID `3`).

Furthermore prepare communication strategies for users and keepers should the limit be approached or reached, explaining why `deposit` or `compound` operations might be temporarily unavailable due to the external Synthetix limit.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Loss Of Rewards When Updating rewardsRegistry

CATEGORY	Rewards
LOCATION	AutoCompoundLP.sol: 471-480
REVIEW	Main Review

DESCRIPTION

This `AutoCompoundLP.setRewardsRegistry` is an `owner-only` function allows changing the address of the `rewardsRegistry` contract. The `AutoCompoundLP` vault uses this registry to claim and value Synthetix rewards associated with its Synthetix `accountId`. The function performs two main actions:

1. Updates the `rewardsRegistry` state variable to the `newRegistry` address.
2. Grants the necessary `REWARDS_PERMISSION` within the Synthetix system to the `newRegistry` address for the vault's `accountId`.

The `setRewardsRegistry` function lacks a mechanism to ensure that any pending rewards, claimable only through the old `rewardsRegistry` address, are claimed before the switch to the `newRegistry` occurs. These pending rewards will become permanently inaccessible (if all the old reward distributors are not registered in the `newRegistry`) via the vault's standard functions listed below:

- `claimRewards`
- `claimSwapCompound`
- `totalAssetsStata` (which internally calls `getUsdValueOfRewards`)

This scenario leads to a potential permanent loss of funds for the vault and its depositors, as the unclaimed rewards become stranded.

RECOMMENDATION

To prevent the potential loss of accrued rewards during a registry update, it is recommended to claim the pending rewards of the old `rewardsRegistry`, before the owner calls `setRewardsRegistry`.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

DoS In Reward Claiming Due To Strict Slippage

CATEGORY	DoS
LOCATION	RewardsDistributionRegistry.sol: 397, 378-382
REVIEW	Main Review

DESCRIPTION

`AutoCompoundLP.sol::claimSwapCompound` calls `RewardsDistributionRegistry.sol::claimRewards` to process rewards.

For synthetic rewards (`rewardIsSynth=true`), `claimRewards` loops through distributors and calls `_unwrapSynth`. This function uses `_unwrappedAmount` to determine the minimum underlying tokens expected (`minAmountReceived`) for the `IWrapperModule.unwrap` call.

`_unwrappedAmount` function calculates this by subtracting exactly `1 wei` from the expected amount, aiming to cover rounding errors. Crucially, this calculation does not factor in the `unwrapFixedFee` specific to the Synthetix spot market (`spotMarketId`) being used for that distributor.

If any distributor's market has an `unwrapFixedFee` configured, the `unwrap` call will receive `expectedAmount - fee`, which is less than the required `minAmountReceived` (`expectedAmount - 1 wei`). This triggers a revert within `IWrapperModule.unwrap` due to failed slippage protection.

Because this check occurs inside the `claimRewards` loop, the failure of even a single distributor's `unwrap` reverts the entire `claimRewards` function call. Consequently, the calling function `claimSwapCompound` also reverts. This leads to a Denial of Service (DoS) vulnerability, preventing keepers from successfully claiming and compounding any rewards system-wide until the problematic distributor is removed or its associated market fee configuration changes.

RECOMMENDATION

Modify the `_unwrapSynth` function (or its helper `_unwrappedAmount`) within `RewardsDistributionRegistry.sol` to correctly calculate the `minAmountReceived` passed to `IWrapperModule.unwrap`, ensuring it accounts for potential fixed unwrap fees. This can be done by querying the `minAmountReceived` amount by calling the `quoteUnwrap` function of the respective `spotMarket`.

L-09 DoS In Reward Claiming Due To Strict Slippage

CONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Unused Token Approvals To ODOS Pose Security Risk

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L327-L328
REVIEW	Main Review

— DESCRIPTION

As per known issues for the vault, SNX is aware that `performSwaps` approves tokens to the ODOS router even if they are not intended to be swapped—specifically SNX, stataUSDC, and USDC.

SNX's position is that the keeper would never attempt to swap these tokens.

However, DEX routers have previously been compromised to exploit unused allowances.

So even if the keeper does not initiate swaps for these tokens, they could still be maliciously swapped with different receivers if the router is compromised.

— RECOMMENDATION

Only approve tokens that are actually required for the intended swap.

— RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Old Rewards Registry Retains Permissions After Replacement

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L475-L476
REVIEW	Main Review

— DESCRIPTION

SNX allows the owner to update the rewards registry if needed.

However, when this change occurs, the `REWARDS_PERMISSION` for the old rewards registry is not revoked.

— RECOMMENDATION

Consider revoking `REWARDS_PERMISSION` from the old rewards registry upon updating to a new one.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

AAVE Deposit/Redeem Limits May Affect stataUSDC Operations

CATEGORY	Error-Handling
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

SNX uses `stataUSDC` as collateral for this vault's account ID.

Each user's USDC deposit is sent to AAVE, which returns `stataUSDC`. This token is then deposited and delegated to SNX.

During redemption, `stataUSDC` is retrieved from SNX Core and then redeemed back to USDC.

Debt repayment also includes the redemption of `stataUSDC` to USDC.

However, AAVE enforces both `maxDeposit` and `maxRedeem` limits:

[Contract Link](https://basescan.deth.net/address/0xf1cd4193bbc1ad4a23e833170f49d60f3d35a621)

These limits could potentially cause reverts during user interactions.

— RECOMMENDATION

While current AAVE limits are unlikely to affect SNX operations, it's important to monitor them.

As TVL grows or if AAVE modifies its caps, vault actions like deposit, redeem, or debt repayment could begin reverting.

Be aware of this possibility and consider implementing safety checks or fallback mechanisms.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Vault Deviates From ERC4626

CATEGORY	Warning
LOCATION	AutoCompoundLP.sol
REVIEW	Main Review

— DESCRIPTION

SNX does not anticipate trading activity involving trading shares, so this is not currently a major issue.

However, both the SNX team and any integrators should be aware that there are different representations of `totalAssets`.

The cached value may not reflect the current state of the vault.

For any external actions involving the vault, depending on the share value, either `sync` functions should be used or `refreshTotalAssetsCache` should be called.

Integrations should be especially cautious when choosing which method to use, and should not assume full ERC4626 equivalence, as this vault implementation is not fully compliant.

— RECOMMENDATION

N/A – This is added as a reference for integrators to consider during implementation.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

RESOLVED

RewardsDistributionRegistry Lacks Flexibility

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/RewardsDistributionRegistry.sol#L363-L364>

REVIEW Main Review

DESCRIPTION

The `RewardsDistributionRegistry` has the USDC value hardcoded as `1e18`.

This can lead to inaccurate share valuations if USDC depegs, as it did briefly in the past.

While SNX's `OracleManager` nodes also configure USDC as a `CONSTANT` node (valued at `1e18`), this can be updated to use a real Pyth price feed if needed.

However, this flexibility does not exist in the `RewardsDistributionRegistry`, where the value is fixed.

RECOMMENDATION

Consider fetching the USDC price from `coreProxy` as well, to ensure dynamic and accurate pricing.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/auto-compound-lp/pull/58>

LOW **L-15**

ACKNOWLEDGED

Issues With Reward Tokens Array Handling

CATEGORY	Unexpected Behavior
LOCATION	RewardsDistributionRegistry.sol
REVIEW	Main Review

DESCRIPTION

The current approach in `RewardsDistributionRegistry` is to add each synth reward underlying token in the `rewardTokens` array and map it to the synth in the `rewardTokenToSynthToken` mapping.

This is done only if the reward token is not already added to the array. Even if the distributor is removed, these data structures are not changes. There are several security consideration around this behavior:

- the call to `getCollateralPrice()` is not wrapped in a `try/catch`, which means if it reverts for an old token, `usdValueOfCallerBalance()` will be DOS-ed
- If the same reward token is added with a different synth, the `rewardTokenToSynthToken` mapping won't be updated and it will keep pointing to the old synth
- If the array becomes too long, looping over it may cause OOG revert

RECOMMENDATION

Reconsider the approach to never delete these values on distributor removal

RESOLUTION

Status: Acknowledged.

LOW **L-16**

RESOLVED

Penalty Rounding In Favor Of The User

CATEGORY Rounding
LOCATION AutoCompoundLP.sol: 149
REVIEW Main Review

DESCRIPTION

The `decayingRedemptionPenalty` function calculates the penalty amount to be deducted during a redemption. The final calculation `assets.mulDiv(timeAdjustedFeePercent, PERCENT_PRECISION)` uses the default behavior of `mulDiv`, which rounds the result (the penalty amount) down.

Rounding the penalty amount down slightly benefits the withdrawing user, as they pay a smaller penalty than if rounding were neutral or upwards. This contradicts the rounding approach potentially used elsewhere, such as when converting shares to assets (e.g., `_convertToAssetsStata` uses `Math.Rounding.Floor`), which explicitly rounds the user's entitled assets down, favoring the protocol.

RECOMMENDATION

To ensure rounding consistently favors the protocol during redemptions involving penalties, modify the final calculation within `decayingRedemptionPenalty` to use ceiling division that rounds the penalty amount up.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/auto-compound-lp/pull/57>

LOW **L-17**

ACKNOWLEDGED

Use `safeApprove()` & `safeTransfer()`

CATEGORY	Best Practices
LOCATION	All uses of <code>transfer()</code> & <code>approve()</code>
REVIEW	Main Review

— DESCRIPTION

The protocol uses `transfer()` and `approve()`, instead of `safeTransfer()` and `safeApprove()`. Some ERC-20 tokens have custom logic, such as USDT on ethereum mainnet's approvals must be reset to zero before creating a new approval. This can lead to unexpected reverts throughout the protocol.

— RECOMMENDATION

Switch all instances of `transfer()` and `approve()` to `safeTransfer()` and `safeApprove()`.

— RESOLUTION

Status: Acknowledged.

LOW **L-18**

RESOLVED

Keeper : Missing Validation On Rewards Return Value

CATEGORY Validation**LOCATION** <https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/main.ts#L86-L87>**REVIEW** Main Review

— DESCRIPTION

The return value of the rewards function is not validated, which could result in an empty rewards list being passed to `inputTokens`.

— RECOMMENDATION

Consider adding validation to check the return value of the rewards function.

— RESOLUTION

Status: Resolved.

LOW **L-19**

RESOLVED

Keeper : getRewards Skips Tokens Held In Vault Balance

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/main.ts#L96-L97
REVIEW	Main Review

— DESCRIPTION

The `getRewards` function only returns newly claimable rewards, and does not account for tokens already held in the vault's balance.

For example, if token A had rewards in the last epoch and was partially swapped—leaving the rest in the vault for periodic swapping—then in the next epoch, if `claimableRewards` for token A is zero, it won't be included in `getRewards`. As a result, the keeper won't attempt to swap it.

— RECOMMENDATION

Consider iterating through the internal rewards array in the keeper script and logging the vault's balance of each token during `populateTokenAmounts`.

— RESOLUTION

Status: Resolved.

LOW **L-20**

RESOLVED

Keeper : Incomplete Error Handling

CATEGORY	Error-Handling
LOCATION	https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/main.ts#L134-L135
REVIEW	Main Review

— DESCRIPTION

The current `get0dosQuote` error handling is incomplete for responses with status codes 422 and 502, as these do not return an `errorCode` in the expected schema.

— RECOMMENDATION

Consider checking for a successful response explicitly, rather than relying on the absence of failure. Revert if a successful return is not detected

— RESOLUTION

Status: Resolved.

LOW **L-21**

RESOLVED

Keeper : assembleOdosQuote May Use Expired Or Invalid Quotes

CATEGORY Error-Handling**LOCATION** <https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/main.ts#L144-L145>**REVIEW** Main Review

— DESCRIPTION

The `assembleOdosQuote` function lacks error handling and does not check for deprecation status.

This could lead to incorrect or expired calldata being used in execution.

— RECOMMENDATION

Consider adding proper error handling in `assembleOdosQuote`, including a check to ensure the quote is not marked as deprecated (the first field in the response).

— RESOLUTION

Status: Resolved.

LOW **L-22**

ACKNOWLEDGED

Keeper : No Monitoring Of Keeper ETH

CATEGORY Error-Handling
LOCATION main.ts
REVIEW Main Review

— DESCRIPTION

There is no tracking for the keeper's ETH balance or the gas used.

This could result in keeper transactions failing due to insufficient funds for gas.

— RECOMMENDATION

Consider adding logs for gas used and remaining ETH balance, along with an alert system to notify admins if the balance falls below a defined threshold.

— RESOLUTION

Status: Acknowledged.

LOW **L-23**

RESOLVED

Keeper : Non-Checksummed Addresses

CATEGORY	Logical Error
LOCATION	config.ts
REVIEW	Main Review

DESCRIPTION

ODOS requires all addresses to be checksummed according to their API documentation:

Parameter	Description	Required
-----------	-------------	----------

---	---	---
-----	-----	-----

tokenAddress	Address of the token to swap from. This should be a [checksummed address] (https://eips.ethereum.org/EIPS/eip-55). Yes
--------------	--

Currently, SNX does not have the following addresses checksummed in `config.ts`, which could result in reverts:

- `0x833589fcd6edb6e08f4c7c32d4f71b54bda02913`
- `0xc1cba3fcea344f92d9239c08c0568f6f2f0ee452`

RECOMMENDATION

Review all addresses used in the keeper infrastructure and ensure they are properly checksummed where required.

RESOLUTION

Status: Resolved.

LOW **L-24**

ACKNOWLEDGED

Keeper : Potential Sandwich Risk

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/config.ts#L40-L41
REVIEW	Main Review

— DESCRIPTION

SNX currently uses Alchemy as their RPC provider.

This setup could become vulnerable to sandwich attacks if Base implements a public mempool with bribes, replacing the current FCFS (First-Come, First-Served) sequencer model.

— RECOMMENDATION

Once Base introduces a public mempool with bribe support, consider switching to a frontrunning-resistant RPC.

— RESOLUTION

Status: Acknowledged.

LOW **L-25**

RESOLVED

Keeper : Default Slippage May Misprice Reward

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/infra/blob/cbd6343f82a985cf1bbd55dbeb304561d1b6f317/packages/lp-vault-keeper/src/util.ts#L18-L19
REVIEW	Main Review

— DESCRIPTION

Since SNX does not use a slippage configuration in `get0dosQuote`, ODOS applies its default slippage of 0.3%.

This may be either too high or too low depending on the specific reward token, potentially leading to suboptimal or failed swaps.

— RECOMMENDATION

Consider adding a slippage parameter in `config.ts` for each reward token and using it when requesting quotes from ODOS.

— RESOLUTION

Status: Resolved.

LOW **L-26**

ACKNOWLEDGED

Migration Omits Synthetix Reward Claiming

CATEGORY Rewards
LOCATION LPMigration.sol: 79-116
REVIEW Main Review

DESCRIPTION

The `migrate` function in `LPMigration.sol` is designed to unwind a user's Synthetix V3 position (collateral and debt) and deposit the net USDC value into the `AutoCompoundLP` vault. However, the current implementation only handles the principal collateral (`$synthUSDC` , `$synthStataUSDC`) and associated debt (`snxUSD`). It completely omits the process of claiming any pending Synthetix rewards accrued by the user's `accountId` for providing liquidity or collateral.

Consequently, the value of these unclaimed rewards is not included in the initial deposit made to the `AutoCompoundLP` vault during the migration. The user must manually claim these rewards later and deposit them separately.

This creates two primary issues:

1. Incomplete Migration & User Burden: The migration is not comprehensive, requiring extra steps from the user.
2. Missed Compounded Yield: The core problem is the delay in depositing the reward value into the `AutoCompoundLP` vault. During this delay, the reward value is not participating in the vault's yield generation strategy (Aave yield + vault's own compounded Synthetix rewards). When the user eventually deposits the reward value, the vault's share price will likely have appreciated due to accrued yield, resulting in the user receiving fewer shares for their rewards than if deposited initially. This represents a loss of potential compounded yield within the target vault for the user.

RECOMMENDATION

Enhance the `migrate` function in `LPMigration.sol` to include the claiming and processing of Synthetix rewards of the migrated position, before depositing into the `AutoCompoundLP` vault.

RESOLUTION

Status: Acknowledged.

LOW **L-27**

ACKNOWLEDGED

Rewards Discount Impacts Share Calculations

CATEGORY Informational
LOCATION AutoCompoundLP.sol: 179-181
REVIEW Main Review

DESCRIPTION

`AutoCompoundLP.totalAssetsStata()` applies `rewardsUsdValueDiscount` to the rewards available to account for fees and slippage when swapping these tokens. Any deposits and redeems executed before the swap of the reward will use the new ratio which includes the discount. Any leftover amount of the discount which was not used for fees will be distributed across the vault participants at the time of the swap, resulting in a profit for the new depositors.

RECOMMENDATION

There is no need for a fix because any amount not used for fees can be considered as a bonus, so documenting the issue and keeping it in mind is enough.

RESOLUTION

Status: Acknowledged.

LOW **L-28**

ACKNOWLEDGED

Tokens With Invalid Prices Are Treated As 0

CATEGORY Unexpected Behavior
LOCATION RewardsDistributionRegistry.sol: 367
REVIEW Main Review

— DESCRIPTION

`RewardsDistributorRegistry.registryDistributor()` doesn't allow adding a distributor if the token it's responsible for doesn't have a valid price, i.e. `getCollateralPrice()` reverts or returns 0.

However, the `_getAssetPrice()` function returns the result of `coreProxy.getCollateralPrice(assetAddress)` as price of the requested asset without doing any checks. If an error occurs at a later point in time inside `getCollateralPrice`, the returned value will be 0 and the given reward token will be treated as 0.

This can lead to unexpected behaviors since users can still interact with the `AutoCompoundLP`, but these rewards tokens will be ignored causing spikes in the value of the total assets.

— RECOMMENDATION

Create a plan for what to do in such emergency cases and implement any changes if needed. Keep in mind that reverting is risky because once a token is added in `rewardTokens`, it stays there forever.

— RESOLUTION

Status: Acknowledged.

LOW **L-29**

ACKNOWLEDGED

Unconditional Rounding In `_repay()`

CATEGORY	Rounding
LOCATION	AutoCompoundLP.sol: 597
REVIEW	Main Review

DESCRIPTION

There is `1` added to `usdcAmount` in `AutoCompoundLP._repay()` to handle rounding issues.

However, the `1` is added unconditionally which creates problems for both `redeem()` and `compound()` because there the amount of `USDC` needed to repay the debt is computed with `mulDiv` and `Math.Rounding.Ceil` which round the number up only if the mathematical result of the division is not a whole number.

For example, consider the case where there is `1e18` debt of the `AutoCompoundLP` position and a user tries to `redeem`. The code will take a flash loan of `1e6` `USDC` in `redeem()`, but will try to wrap `1e6 + 1` because of this `+1` happening, resulting in reverting `redeem` transaction. The same is true for the `compound()` function.

RECOMMENDATION

Replace the [rounding](https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L595-L597) with `mulDiv`.

After that, you can delete the following [-1](https://github.com/GuardianOrg/auto-compound-lp-team1/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/AutoCompoundLP.sol#L372)

RESOLUTION

Status: Acknowledged.

LOW **L-30**

ACKNOWLEDGED

Cached Assets Don't Account For Redeem Penalty

CATEGORY Math
LOCATION AutoCompoundLP.sol: 270
REVIEW Main Review

DESCRIPTION

When users redeem their `AutoCompoundLP` shares, `_convertToAssetsStata()` will update the `totalAssetsCache` to `currentTotalAssets - assets`. Here `assets` is the requested amount to be redeemed, but it doesn't include the penalty amount which technically doesn't leave the system and should not be subtracted. This results in a discrepancy between the real USDC value and what's cached and in turn leads to wrong results returned from `previewDeposit` and `previewRedeem()`.

RECOMMENDATION

You can make the `refreshTotalAssetsCache()` visibility public and use it throughout the codebase to refresh the cache.

RESOLUTION

Status: Acknowledged.

LOW **L-31**

ACKNOWLEDGED

Liquidations Break The Vault

CATEGORY Unexpected Behavior
LOCATION AutoCompoundLP.sol
REVIEW Main Review

DESCRIPTION

The `AutoCompoundLP` vault is a participant in the v3 core system which means it's a subject to liquidations if its collateral ratio becomes unhealthy. If that happens, both its debt and collateral will be erased, but the vault depositors will continue holding all their previous tokens. If the vault remains active after the liquidation, the old depositors will have exposure to all the funds deposited afterwards, leading to loss for the new depositors and profit for the old ones.

RECOMMENDATION

Consider halting the vault in case its experiences a liquidiation.

RESOLUTION

Status: Acknowledged.

LOW **L-32**

ACKNOWLEDGED

Donator Can Undelegate Vault Collateral

CATEGORY Informational
LOCATION AutoCompoundLP.sol
REVIEW Main Review

— DESCRIPTION

When calculating the worth of the `AutoCompoundLP` shares, the reward tokens held in the contract are also included in `totalAssetsStata`. This allows a vault participant to donate a large amount of rewards to the vault, change the share price and then redeem their shares for a large amount of the current delegated assets, leaving the vault with only `minDelegationAmount` or even `0` assets.

Doing this is expensive since the more assets they are in the vault, the more rewards the user has to donate, but it can lead to some unexpected behaviors, like the `AutoCompoundLP` not receiving rewards and liquidation rewards or depositors not being able to redeem because there is not enough liquidity.

— RECOMMENDATION

Document this behavior.

— RESOLUTION

Status: Acknowledged.

LOW **L-33**

ACKNOWLEDGED

Locked Market Can Block Redeem

CATEGORY Informational
LOCATION AutoCompoundLP.sol
REVIEW Main Review

— DESCRIPTION

When users redeem, if there is debt accrued by the `AutoCompoundLP` position, it will be repaid with an Aave flashloan.

Then the amount undelegated from the position is the amount to be received from the user + the amount to repay the flashloan.

This can cause the amount delegated to the market to fall below its `minimumCredit` and in result the redeem will be unsuccessfull.

— RECOMMENDATION

Document the risk.

— RESOLUTION

Status: Acknowledged.

LOW **L-34**

ACKNOWLEDGED

Unnecessary Reward Operation

CATEGORY Informational
LOCATION RewardsDistributionRegistry.sol
REVIEW Main Review

DESCRIPTION

`RewardsDistributionRegistry.claimRewardsFromDistributor()` and `RewardsDistributionRegistry.getRewards()` call `claimPoolRewards()` and `getAvailablePoolRewards()` twice - the first time for the `lpCollateralAsset` vault and the second time for the `address(0)` vault. Usually, it's not expected to have active stakers in the vault for `address(0)` collateral, which makes the second call unnecessary.

RECOMMENDATION

Consider whether the second call is necessary and remove it otherwise.

RESOLUTION

Status: Acknowledged.

LOW **L-35**

RESOLVED

Alchemy API Key Exposed

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/auto-compound-lp-team2/blob/19433ae94f66e15a749dfab3e61d0378064aaa79/contracts/test/LPMigration.t.sol#L45-L46>

REVIEW Main Review

DESCRIPTION

The API key for your Alchemy account is exposed in your test cases. This can allow anyone to use your API and make calls from your account.

RECOMMENDATION

Use a `.env` file to store the API key. Then concatenate a string with the base of the URL with the API key.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/auto-compound-lp/pull/57>

LOW **L-36**

ACKNOWLEDGED

Keeper : Dead Code / TODOs

CATEGORY Best Practices
LOCATION main.ts / utils.ts
REVIEW Main Review

— DESCRIPTION

Functions like `getRandomVariance()` and `submitTransaction()` are stubs; TODOs indicate unfinished logic that can mask security gaps.

— RECOMMENDATION

Remove or complete TODOs; run static analysis to ensure no unreachable code remains.

— RESOLUTION

Status: Acknowledged.

LOW **L-37**

ACKNOWLEDGED

Keeper : Hard-coded Resource Identifiers

CATEGORY	Best Practices
LOCATION	config.ts
REVIEW	Main Review

— DESCRIPTION

Token lists, chain IDs, and account-specific constants are fixed in code, causing config drift across environments.

— RECOMMENDATION

Externalise to SSM parameters/CDK context and document overrides for staging vs. prod.

— RESOLUTION

Status: Acknowledged.

LOW **L-38**

ACKNOWLEDGED

Keeper : Non-optimal Lambda Sizing

CATEGORY Best Practices
LOCATION lp-vault-keeper.ts
REVIEW Main Review

— DESCRIPTION

`memorySize:256 MB` with `timeout:30 s` could be over- or under-provisioned, affecting cost and latency.

— RECOMMENDATION

Benchmark under load; right-size memory (which also boosts CPU) and reduce timeout to ≤ 10 s if feasible.

— RESOLUTION

Status: Acknowledged.

LOW **L-39**

ACKNOWLEDGED

Keeper : Opaque Cross-Stack Imports

CATEGORY	Best Practices
LOCATION	lp-vault-keeper.ts
REVIEW	Main Review

— DESCRIPTION

Subnet IDs/security group IDs are imported via CloudFormation outputs without tags or documentation, hindering troubleshooting.

— RECOMMENDATION

Tag imported resources (`cdk.Tags.of(...)`) and maintain design-docs/diagrams that map stack dependencies.

— RESOLUTION

Status: Acknowledged.

LOW **L-40**

ACKNOWLEDGED

Keeper : Sensitive Request/Response Logging

CATEGORY	Best Practices
LOCATION	lp-vault-keeper.ts
REVIEW	Main Review

DESCRIPTION

`dataTraceEnabled` is true (non-prod), writing full request/response bodies—including tokens & calldata—to CloudWatch.

RECOMMENDATION

Disable full traces or mask fields with log-filtering. Encrypt logs and limit reader IAM roles.

RESOLUTION

Status: Acknowledged.

LOW **L-41**

RESOLVED

Keeper : Short Log Retention

CATEGORY	Best Practices
LOCATION	lp-vault-keeper.ts
REVIEW	Main Review

— DESCRIPTION

Logs are retained for ONE_WEEK — insufficient for many compliance or forensic requirements.

— RECOMMENDATION

Extend retention to ≥ 90 days or export to S3 + Glacier automatically.

— RESOLUTION

Status: Resolved.

LOW **L-42**

RESOLVED

Keeper : Inefficient Secret Retrieval

CATEGORY Best Practices
LOCATION utils.ts / main.ts
REVIEW Main Review

DESCRIPTION

`SecretsManager.getSecretValue()` is executed on every invocation, adding latency and cost.

RECOMMENDATION

Cache the secret in /tmp (persist across warm invocations) or use Secrets Manager extension.

RESOLUTION

Status: Resolved.

LOW **L-43**

RESOLVED

Keeper : RemovalPolicy DESTROY On Logs

CATEGORY Best Practices
LOCATION lp-vault-keeper.ts
REVIEW Main Review

DESCRIPTION

`RemovalPolicy.DESTROY` deletes log groups when the stack is torn down, erasing evidence needed for investigations.

RECOMMENDATION

Set `RemovalPolicy.RETAIN` in production and automate log archival to S3 before stack deletion.

RESOLUTION

Status: Resolved.

LOW **L-44**

RESOLVED

Keeper : External Calls Lack Timeout & Retry

CATEGORY Best Practices
LOCATION utils.ts
REVIEW Main Review

— DESCRIPTION

`fetch()` to Odos/Alchemy has no timeout or retry; slow responses lead to Lambda timeouts.

— RECOMMENDATION

Wrap calls with AbortController (≤ 5 s) and implement exponential back-off with jitter.

— RESOLUTION

Status: Resolved.

LOW **L-45**

RESOLVED

Keeper : Missing Input Validation

CATEGORY	Validation
LOCATION	Handler Code
REVIEW	Main Review

— DESCRIPTION

Proxy integration forwards raw payloads; the Lambda never validates schema or data types, enabling injection or resource-exhaustion attacks.

— RECOMMENDATION

Add strong JSON schema validation (e.g., `ajv`) and early-return `400` on malformed requests.

```
if(!event.body) return {statusCode:400,body:'Missing body'};
```

— RESOLUTION

Status: Resolved.

LOW **L-46**

RESOLVED

Keeper : Over-Permissive / Implicit Lambda Role

CATEGORY	IAM
LOCATION	lp-vault-keeper.ts
REVIEW	Main Review

DESCRIPTION

No explicit IAM role is attached; CDK's default role may include permissions the function doesn't need or omit required ones.

RECOMMENDATION

Define a custom least-privilege role

```
const lambdaRole = new iam.Role(this, 'Role', {assumedBy: new
iam.ServicePrincipal('lambda.amazonaws.com')});
lambdaRole.addManagedPolicy(iam.ManagedPolicy.fromAwsManagedPolicyName('service-
role/AWSLambdaVPCLambdaAccessExecutionRole'));
lambdaRole.addToPolicy(new iam.PolicyStatement({actions:
['secretsmanager:GetSecretValue'], resources: ['*']}));
```

RESOLUTION

Status: Resolved.

LOW **L-47**

ACKNOWLEDGED

redeem Revert Due To Flash Loan Underfunding

CATEGORY	DoS
LOCATION	AutoCompoundLP.sol: 277, 591-592
REVIEW	Main Review

DESCRIPTION

The `redeem` function calculates the amount of `USDC` needed for an Aave flash loan (`usdcNeededToRepay`) to repay the user's Synthetix debt (`debt`). This calculation is performed by the `usdcToRepayDebt` helper function.

Currently, `usdcToRepayDebt` performs a direct 1:1 decimal conversion from the 18-decimal `debt` (denominated in `snxUSD`) to 6-decimal `USDC` . However, the actual repayment happens within the Aave flash loan callback (`executeOperation`), specifically inside the `_repay` internal function. This function determines the precise amount of `synthUSDC` needed to acquire the target `debtAmount` of `snxUSD` by querying the Synthetix market via `IAtomicOrderModule.quoteSellExactOut` .

Crucially, the quote returned by `quoteSellExactOut` *includes potential positive `skewFees` * imposed by the Synthetix market if the trade would increase market imbalance.

If a positive `skewFee` is applied by Synthetix during the `quoteSellExactOut` call, the required `synthAmount` (and consequently the `usdcAmount` needed in `_repay`) will be slightly higher than the simple 1:1 conversion amount calculated initially by `usdcToRepayDebt` when determining the flash loan size (`usdcNeededToRepay`). As a result the contract will *lack sufficient `USDC` * received from the flash loan to perform the necessary `wrap` operation inside `_repay` .

This results in the entire `redeem` transaction reverting due to insufficient funds during the flash loan callback execution.

RECOMMENDATION

To guarantee the flash loan always covers the required repayment amount, including any potential Synthetix `skewFees` , modify the `redeem` function to calculate `usdcNeededToRepay` based on a real-time quote from the Synthetix market before initiating the flash loan. Else recommended to apply enough buffer on the `usdcNeededToRepay` value before calling the `flashloan` such that the `_repay` will not revert due to insufficient funds.

L-47 redeem Revert Due To Flash Loan Underfunding

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-48**

RESOLVED

Keeper :Outdated AWS CDK And Deprecated SDK

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

In the [`lp-vault-keeper.ts`](<https://github.com/GuardianOrg/infra/blob/main/infra/cdk/lib/lp-vault-keeper/lp-vault-keeper.ts>) file, the script relies on `aws-cdk-lib`.

However, the version used (as per [`package.json`](<https://github.com/GuardianOrg/infra/blob/main/infra/cdk/package.json>)) is `2.186.0`, which is affected by the 2L/1M vulnerability.

Additionally, the [`lp-vault-keeper` package](<https://github.com/GuardianOrg/infra/blob/main/packages/lp-vault-keeper/package.json#L14>) lists `aws-sdk` as a dependency. This SDK has been in maintenance mode since September 2024 and will no longer receive full support beyond September 2025.

This introduces long-term maintenance and potential security risks, especially for components relying on AWS services like Cognito, IAM, or Lambda integrations.

RECOMMENDATION

- Upgrade `aws-cdk-lib` to the latest secure version (e.g., `2.191.0` as of the time of this review).
- Migrate from `aws-sdk` (v2) to the modular v3 (`@aws-sdk/*`) packages, as recommended by AWS.

This ensures continued compatibility, improved performance, and access to ongoing security patches.

RESOLUTION

Status: Resolved.

LOW **L-49**

ACKNOWLEDGED

Trust Assumptions

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

**** Rewards Registry Decimal Precision****

The current version of the rewards registry is not expected to handle reward tokens with fewer than 18 decimals. All configured synth rewards are standardized to 18 decimals.

**** Pricing, Oracles & Precision Conversion Ratios****

Wrap/unwrap operations for USDC, snxUSD, and sUSD are assumed to follow a 1:1 value ratio with no fee.

However, minor rounding deltas are known and are currently handled via manual **+1/-1** adjustments in logic.

Oracle Pricing and Staleness Risk

The price of synths is derived from their underlying collateral via Pyth oracles.

It is assumed prices will generally remain fresh, as SNX uses a keeper that updates all markets hourly and key assets (e.g., ETH, BTC) at price thresholds.

However, temporary staleness remains a valid risk.

sStataUSDC Precision Assumption

It is assumed that the price of **sStataUSDC** will consistently end in 12 zeros (fixed-point format).

This has been validated through testing but is not guaranteed. Deviation could result in rounding or precision loss during conversions.

**** Keeper Behavior & Management Trusted Keeper Execution****

It is assumed that the keeper will consistently and honestly perform its upkeep duties.

Secure Keeper Key Management

L-49 Trust Assumptions

CONTINUED

The keeper operates as an EOA. It is assumed that key access is tightly controlled and will not be misused by insiders or compromised from developer machines.

Low Risk of Insider Sandwiching Attacks

It is assumed that knowledge of pending compound events will not lead to effective sandwich attacks.

This is mitigated by randomized execution timing and Odos routing.

Randomized Keeper Timing Prevents Predictability

Random variance in keeper execution is assumed to reduce predictability and MEV risks during compound swaps.

Swap Size Limits Prevent Exploitable Volume

Maximum swap sizes ensure that large swaps are broken into smaller chunks, reducing the risk of slippage or sandwiching attacks.

Separation of Claim, Swap, and Compound Logic

These actions are modular in the keeper, allowing finer control and flexibility to avoid swapping dust tokens or executing unneeded compound steps.

** Vault Entry/Exit & User Behavior Withdrawal Penalty Discourages Quick-In/Quick-Out**

A decaying withdrawal penalty (e.g., 1% over 1 day) is assumed to discourage users from quickly entering and exiting the vault in response to large trader liquidations.

24-Hour Transfer Lock Prevents Penalty Circumvention

Vault shares are non-transferable for 24 hours after deposit.

This is assumed to prevent circumvention of withdrawal penalties via share transfer to another address.

Timeout Overrides During Migration

- For the LP vault, `_CONFIG_SET_ACCOUNT_OVERRIDE_WITHDRAW_TIMEOUT` will be used.
- For migration contracts, `_CONFIG_SET_SENDER_OVERRIDE_WITHDRAW_TIMEOUT` is planned (not yet deployed on Base V3).

These allow per-user or per-sender timeout bypassing when needed.

TotalAssetsSync Includes Uncompounded Rewards

Uncompounded rewards are reflected in `totalAssetsSync`.

L-49 Trust Assumptions

CONTINUED

It is assumed this may introduce temporary valuation mismatch during mass redemptions, but is acceptable due to flexibility in triggering compounding.

**** Partial ERC4626 Compliance Deviation from ERC4626 Interface ****

The vault does not fully implement the ERC4626 standard.

Functions such as `totalAssets` may require state-changing operations and are not strict `view` functions.

It is assumed that integrators are aware of this and will use the provided alternatives (e.g., `totalAssetsSync`) instead of expecting strict interface conformance.

— RECOMMENDATION

Beware of these trust assumptions, a change in any of these may require a code change.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Stuck Rewards in RewardsDistributionRegistry

CATEGORY	Logical Error
LOCATION	RewardsDistributionRegistry.sol: L194-L199
REVIEW	Remediation Review

DESCRIPTION

The `claimRewardsFromDistributor` function in `RewardsDistributionRegistry.sol` is currently `public`. While an access control check (`if (msg.sender != coreProxy.getAccountOwner(accountId)) revert UserNotAccountOwner();`) was added to ensure only the account owner can initiate the claim from the Synthetix `coreProxy`, severe issue remains.

If an account owner calls `claimRewardsFromDistributor` directly, the rewards are claimed from the `coreProxy` and are held by the `RewardsDistributionRegistry` contract. However, the primary `claimRewards` function is designed to process and transfer rewards that are claimed during its own transaction execution.

Specifically, `claimRewards` iterates through distributors, calls `claimRewardsFromDistributor` for each, and then transfers the `totalClaimed` amount (which is the result of this specific internal call) to `msg.sender`. If `claimRewardsFromDistributor` was called directly in a previous, separate transaction, the rewards from that initial call are now sitting in the `RewardsDistributionRegistry`. When `claimRewards` is subsequently called, `getRewardsFromDistributor` will likely return 0 for that distributor.

As a result, the rewards claimed through a direct, prior call to the public `claimRewardsFromDistributor` function are never transferred to the user by the `claimRewards` function and become permanently stuck in the `RewardsDistributionRegistry` contract.

RECOMMENDATION

Move the check back to the old function (for gas savings) and change the visibility of `claimRewardsFromDistributor` to `internal`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Keeper Ineffective /tmp Caching Logic Causes Crash on Cold Start

CATEGORY IAM
LOCATION util.ts
REVIEW Remediation Review

— DESCRIPTION

SNX introduced a `/tmp` check so the Lambda looks for `private_key.txt` and `rpc_url.txt` before calling Secrets Manager. That's the right idea, but the implementation is incomplete.

What SNX did:

- `getWallet` and `getRpcUrl` read the file in `/tmp`; if it exists, they use it. Otherwise, they fall back to Secrets Manager.

Remaining problems:

1. The initial `fs.readFile` isn't wrapped in `try-catch`. On the first cold start, the file won't exist, `fs` throws `ENOENT`, and the whole invocation crashes before Secrets Manager is even reached.
2. When SNX does fetch the secret, it never writes it back to `/tmp`, so every new container hits Secrets Manager again. This keeps the latency and the extra API charge that was supposed to be avoided.

Why this matters:

A cold-start crash means the keeper skips a cycle. Repeated Secrets Manager calls add 100ms+ to cold starts and burn through the free tier.

— RECOMMENDATION

Wrap `fs.readFile` in a `try-catch` that suppresses only "file not found." After fetching the secret, immediately write it to `/tmp` with permissions `0600` so warm invocations can hit the cache. Maintain a module-level variable to prevent repeated file system access in the same invocation.

****Ensure the file written to `/tmp` is encrypted.****

M-02 Keeper Ineffective /tmp Caching Logic Causes Crash on Cold StartCONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Keeper Wildcard SecretsManager Policy

CATEGORY Access Control
LOCATION lp-vault-keeper.ts
REVIEW Remediation Review

DESCRIPTION

A custom IAM role was introduced (good) but granted `secretsmanager:GetSecretValue` on `*`. If an attacker gets Lambda code-exec they can read any secret in the account.

RECOMMENDATION

Scope the policy to the two specific secret ARNs (private key and RPC URL) or use a dedicated Secrets Manager resource policy.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Interface Mismatch for migrateLP Signature

CATEGORY	Logical Error
LOCATION	LPMigration.sol#L79
REVIEW	Remediation Review

DESCRIPTION

The `LPMigration.sol` contract's `migrate` function was updated to address a previous vulnerability (High-01: Anyone can update users userLastDeposited). This remediation involved removing the `receiver` parameter from the `migrate` function, and instead, the `msgSender` (derived via `ERC2771Context._msgSender()`) is now implicitly used as the recipient of the `AutoCompoundLP` tokens.

However, the `ILPMigration.sol` interface still defines the `migrateLP` function with the `receiver` parameter:

```
function migrateLP(uint128 accountId, address receiver) external;
```

This creates a discrepancy between the interface definition and the actual implementation in `LPMigration.sol`.

RECOMMENDATION

To maintain consistency and clarity, the `migrateLP` function signature in the `ILPMigration.sol` interface should be updated to match the implemented `migrate` function in `LPMigration.sol`.

Also consider inheriting `ILPMigration` in `LPMigration`.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

L-16 Partially Fixed : Rounding Penalty

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/auto-compound-lp-team1/blob/d6dd42f3736d81e1406e1ac6a026b831a4e5989e/contracts/AutoCompoundLP.sol#L126-L127
REVIEW	Remediation Review

— DESCRIPTION

Division is still rounded down here

```
function decayingRedemptionPenalty(  
    address user,  
    uint256 assets  
) public view returns (uint256) {  
    -----  
    return  
    assets.  
    mulDiv(timeAdjustedFeePercent,  
    PERCENT_PRECISION); <<<<<<<  
}
```

— RECOMMENDATION

Consider rounding up for both of above divisions

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Inconsistent Calculations in case of USDC depeg.

CATEGORY Logical Error
LOCATION RewardsDistributionRegistry.sol
REVIEW Remediation Review

DESCRIPTION

`RewardsDistributionRegistry.getUsdValueOfRewards()` returns the value of the available rewards in `USDC`, not `USD` (even though the name says `Usd`). The v3 debt is being subtracted from the returned value. This will result in wrong calculations in case of a USDC depeg because the v3 debt is in `USD` while the returned value from the function is in `USDC`.

RECOMMENDATION

Consider modifying `getUsdValueOfRewards` to return the value in `USD`. If you do that, you should convert the `absAmount` back to `USDC` in `AutoCompoundLP.totalAssetsStata()` before dividing it by the static token `rate`.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Unfixed H-03

CATEGORY	Warning
LOCATION	RewardsDistributionRegistry.sol
REVIEW	Remediation Review

— DESCRIPTION

The [H-03](https://www.notion.so/guardianaudits/H-03-1dd8bda5828c80f296ffc52aadab791a?pvs=4) finding from the main review was not fixed because it's expected that rewards will always be distributed in `synths`. However, given that the fix is easy and the impact of the issue would not be negligible if normal tokens were distributed as rewards, it's recommended to fix it, since the other part of code is already written to support this case.

— RECOMMENDATION

Consider implementing the fix from the original issue.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Alchemy API key can still be accessed

CATEGORY Logical Error
LOCATION Repository
REVIEW Remediation Review

DESCRIPTION

The `Alchemy` API key was moved from the `AutoCompoundLP.t.sol` and the other test files to an env, but it is still visible in the repository commit history.

RECOMMENDATION

Consider deprecating this api key

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

ODOSRouter Max Approvals : Risk Increased

CATEGORY Warning
LOCATION AutoCompoundLP.sol
REVIEW Remediation Review

— DESCRIPTION

As mentioned in [L-10](<https://www.notion.so/guardianaudits/L-10-1df8bda5828c800da46cc0127fdb023f?pvs=4>), leaving approvals for the ODOS router poses security risks in case the router is exploited. The team acknowledged the issue, but after that the reward approvals to the ODOS router were changed to `uint256.max`. This decision makes the situation riskier.

— RECOMMENDATION

Consider not using max approvals if they are not really needed.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Keeper Invalid handler signature

CATEGORY	Logical Error
LOCATION	main.ts
REVIEW	Remediation Review

DESCRIPTION

According to the [AWS documentation](https://docs.aws.amazon.com/lambda/latest/dg/typescript-handler.html), the `handler()` function should accept parameters of `S3Event` and `Context` types. This was true in the [previous version](https://github.com/GuardianAudits/snx-infra2-archive/blob/main/packages/lp-vault-keeper/src/main.ts#L32) of the code. However, with the latest changes, the handle function was modified to accept a boolean value, which is currently more suited for local dev than for prod use in AWS lambda environment.

RECOMMENDATION

Check the documentation and revert back to the old signature in case it's needed.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Keeper RPC-Key is still hardcoded

CATEGORY	Warning
LOCATION	config.ts
REVIEW	Remediation Review

DESCRIPTION

In M-07, SNX said that `SECRET_ARN` is just unique identifier used to reference private key, so they are ok with it, but the `RPC_URL` shouldn't be publicly visible and "held separately". However, the same url is still exposed in [config.ts]
(<https://github.com/GuardianOrg/infra/blob/77649ed95e5620411d2366b57dd99b06481ac6a9/packages/lp-vault-keeper/src/config.ts#L63>)

RECOMMENDATION

Ensure to implement the "To-Do", as stated in the comments, and avoid exposing the RPC URL. Keep in mind that it will still be visible in the commit history, hence its better to deprecate this key and use a fresh one.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Keeper Slippage Tolerance Not Passed to ODOS in Quote Request

CATEGORY	Logical Error
LOCATION	utils.ts L26
REVIEW	Remediation Review

— DESCRIPTION

As a fix for L-25, SNX has added slippage tolerance within its config. However, this value is not being passed when quoting via ODOS:

```
const requestBody: QuoteRequest = {
  chainId: 8453,
  compact: true,
  inputTokens: rewardTokens,
  outputTokens: [
    {
      tokenAddress: USDC,
      proportion: 1,
    },
  ],
  userAddr: slpAddr,
};
```

According to [ODOS API documentation](https://docs.odos.xyz/build/api-docs), slippage can be configured using the `slippageLimitPercent` field in the request body.

— RECOMMENDATION

Include the configured slippage tolerance by setting the `slippageLimitPercent` field in the request body. Note that ODOS supports only a single common slippage configuration for all tokens. (hence consider the most conservative one)

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Keeper L21 Partially Fixed: Missing Error Handling for assemble0dosQuote Response

CATEGORY Validation
LOCATION main.ts L119
REVIEW Remediation Review

DESCRIPTION

As a fix for L-21, SNX has added a deprecation check. However, it still does not handle status codes returned from `assemble0dosQuote`.

RECOMMENDATION

Consider adding explicit error handling for the `assemble0dosQuote` call.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Keeper Missing Input Validation

CATEGORY Validation
LOCATION handler (main.ts)
REVIEW Remediation Review

DESCRIPTION

L-45 is marked as resolved, however proxy API still forwards raw requests and the Lambda does no schema checks. An internal caller sending malformed JSON could crash the function or cause expensive loops.

RECOMMENDATION

Add JSON schema validation (e.g. AJV) and early-return 400 on bad input to keep failures cheap.

RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

Keeper All addresses aren't checksummed

CATEGORY	Logical Error
LOCATION	config.ts
REVIEW	Remediation Review

DESCRIPTION

ODOS requires all addresses to be checksummed according to their API documentation. However all the addresses aren't checksummed.

There are several addresses, which are yet to be changed in `config.ts`:

`cbBTC`, `cbETH`, `SNX`

RECOMMENDATION

Replace the non-checksum addresses with checksum addresses :

```
const SNX = '0x22e6966B799c4D5B13BE962E1D117b56327FDa66';  
const cbETH = '0x2Ae3F1Ec7F1F5012CFEab0185bfc7aa3cf0DEc22';  
const cbBTC = '0xcbB7C0000aB88B473b1f5aFd9ef808440eed33Bf';
```

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.